Week 2 - AMI pipelines with EC2 Image Builder

DOP-C02 Domain 1 (SDLC Automation) and Domain 2 (Configuration Management and IaC). Exam focus: golden AMIs and immutable infrastructure, Image Builder components, recipes, infrastructure and distribution configurations, pipeline schedules, testing and scanning, rolling out new AMIs, and the common failure causes.

Part 1 - Why AMI pipelines (golden images)

A **golden AMI** is a pre-built, patched, hardened machine image with your agents and base software already installed. Instead of configuring every server after boot, you **bake** the configuration into the image and launch identical instances from it.

Benefit	Why it matters for DevOps
Immutable infrastructure	Replace instances with a new AMI instead of patching in place: no configuration drift
Faster scaling	Software is pre-installed, so Auto Scaling instances are ready in seconds
Consistent environments	Dev, test and prod run the same image version
Security and compliance	OS patches, CIS / STIG hardening and vulnerability scanning happen **before** deployment
Auditable versions	Every image has a version and build history; rollback = launch the previous AMI

- **Ways to build AMIs: EC2 Image Builder** (managed, the exam answer for least operational overhead), **HashiCorp Packer** (often in CodeBuild), or manual launch-configure-create image (not repeatable).
- **Bake vs bootstrap:** bake slow, stable things (OS patches, runtimes, agents, hardening); bootstrap fast, environment-specific things at launch (user data, Parameter Store config, secrets fetched at runtime).
- **Never bake secrets** into an AMI.

Part 2 - Image Builder building blocks

```
Components (build + test steps) --+
Parent image ---------------------+--> Image recipe --+
Infrastructure configuration -------------------------+--> Image pipeline (schedule / manual / API)
Distribution configuration --------------------------+          |
                                                                v
                                                  Image (AMI or container) --> Distribution
```

1. Components (AWSTOE documents)

- A **YAML document** run by **AWSTOE** (AWS Task Orchestrator and Executor) on the build and test instances.
- **Phases:**
  - `build`: install and configure software on the build instance.
  - `validate`: check the build on the **same build instance**, before the AMI is created.
  - `test`: run on a **new test instance launched from the new AMI**.
- **Action modules:** `ExecuteBash`, `ExecutePowerShell`, `ExecuteBinary`, `S3Download`, `WebDownload`, `UpdateOS`, `Reboot`, `InstallMSI`, `CreateFile`, `SetRegistry` and others.
- **Semantic versioning** (`1.0.0`). A component version is **immutable**: changes need a new version.
- **Parameters** make one component reusable with different inputs.
- **AWS-managed components:** `update-linux` / `update-windows`, `amazon-cloudwatch-agent-linux`, `aws-cli-version-2-linux`, `stig-build-linux-high`, `reboot-test-linux`, `simple-boot-test-linux`, `inspector-test-linux`.
- Components can be **shared** with other accounts (AWS RAM) or come from **AWS Marketplace**.

```yaml
name: install-nginx
schemaVersion: 1.0
phases:
  - name: build
    steps:
      - name: InstallNginx
        action: ExecuteBash
        inputs:
          commands:
            - sudo dnf install -y nginx
            - sudo systemctl enable nginx
  - name: validate
    steps:
      - name: CheckBinary
        action: ExecuteBash
        inputs:
          commands:
            - nginx -v
  - name: test
    steps:
      - name: ServiceRunsOnBoot
        action: ExecuteBash
        inputs:
          commands:
            - systemctl is-active nginx
            - curl -sf http://localhost/ > /dev/null
```

2. Image recipe

- **Parent (base) image:** an AMI ID, an **AWS-managed image** (for example Amazon Linux 2023), an **SSM parameter** holding an AMI ID, or a Marketplace / **CIS-hardened** image. A version filter such as `x.x.x` always uses the **latest** parent.
- An **ordered list of components** (build and test) with parameters.
- **Block device mappings:** root volume size and type, extra volumes, encryption.
- Working directory, optional user data override, option to **remove the SSM agent** after the build.
- **Recipes are versioned and immutable**, like components.

- **Container recipe:** parent Docker image plus a **Dockerfile template** with placeholders such as `{{{ imagebuilder:parentImage }}}` and `{{{ imagebuilder:components }}}`, plus the target **ECR repository**. Image Builder builds **container images too**.

3. Infrastructure configuration

Setting	Notes
Instance profile	**Must include** `AmazonSSMManagedInstanceCore` (Image Builder drives the instance through **Systems Manager**) and `EC2InstanceProfileForImageBuilder`. Add S3 read and ECR permissions as needed
Instance types	One or more. Builds can use a larger type than production
Subnet and security group	**Outbound access to SSM, S3 and package repositories** through a **NAT gateway or VPC endpoints** (ssm, ssmmessages, ec2messages, S3 gateway). **No inbound ports** needed
Key pair	Optional. Not needed because SSM is used
Logging	Build logs to **S3**; also **CloudWatch Logs** (`/aws/imagebuilder/...`)
SNS topic	Notifications on build status
`terminateInstanceOnFailure`	Default **true**. Set **false** to keep a failed build instance for troubleshooting (Session Manager)
Instance metadata options	Require **IMDSv2**
Resource tags	Tags for build instances (cost, policy)

4. Distribution configuration

- **Regions:** copy the AMI to several Regions in one step.
- **Target accounts: launch permissions** for accounts, **organizations or OUs** (sharing), or **copy** into other accounts (target accounts need `EC2ImageBuilderDistributionCrossAccountRole`).
- **AMI name** with variables such as `{{ imagebuilder:buildDate }}`, plus AMI tags.
- **KMS key per Region** for encryption. Cross-account: the **key policy must allow the target accounts** and their Auto Scaling service-linked role.
- **Launch template update:** set the new AMI in a launch template, optionally as the **default version**, in this or another account.
- **SSM parameter update:** write the new AMI ID to **Parameter Store**, for example `/golden/al2023/latest`.
- **Windows fast launch**, **License Manager** configuration, **S3 export** (VMDK, VHD or RAW).
- **Container images:** distribute to **ECR** repositories in other Regions.

5. Image pipeline

- Combines a **recipe**, an **infrastructure configuration** and a **distribution configuration**.
- **Trigger: manual**, **API** (`StartImagePipelineExecution` from Lambda, CodePipeline or EventBridge), or a **schedule** (cron).
- **Schedule start condition:**
  - `EXPRESSION_MATCH_ONLY`: build on every schedule run.
  - `EXPRESSION_MATCH_AND_DEPENDENCY_UPDATES_AVAILABLE`: build **only if** the **parent image or a component has a newer version**.
- **Image tests** on or off, with a timeout.
- **Image scanning with Amazon Inspector**: findings per image in the console.
- **Enhanced metadata collection** (package inventory through SSM).
- **Workflows:** custom build, test and distribution steps. `WaitForAction` pauses the build until a Lambda or a person resumes it: **approval gates** and external tests.

6. Images, versions and lifecycle

- Image identity: `name/version/build`, for example `golden-al2023/1.2.0/3`.
- **States:** PENDING, BUILDING, TESTING, DISTRIBUTING, **AVAILABLE**, or **FAILED** / CANCELLED.
- **Image lifecycle policies:** automatically **deprecate**, **disable** and **delete** old AMIs, their **snapshots**, and old container images.
  - Rules by **count** (keep last N), **age**, **tags**, and **exclusion rules** (for example keep `release=prod`).
  - **Deprecated** AMIs are hidden from new launches by default but still usable.

Part 3 - What happens during a pipeline run

1. Image Builder starts a **Systems Manager Automation** workflow.
2. It **launches a build instance** from the parent image using the infrastructure configuration.
3. **AWSTOE** runs each component's **build** phase, then **validate** (`update-linux` applies OS patches here).
4. The instance is **stopped** and an **AMI (with snapshots)** is created.
5. A **test instance** is launched **from the new AMI** and each component's **test** phase runs.
6. **Inspector** scans the image if scanning is on.
7. **Distribution:** copy, share or encrypt per Region and account; update launch templates and SSM parameters.
8. Build and test instances are **terminated** (unless `terminateInstanceOnFailure` is false and the run failed).
9. **Events: EventBridge** "EC2 Image Builder Image State Change" (AVAILABLE / FAILED), **SNS** notification, **CloudWatch Logs / S3** logs.

Part 4 - Rolling out new AMIs (CI/CD integration)

Getting the new AMI ID to consumers

Method	How
SSM Parameter Store (common exam answer)	Distribution (or a Lambda on the EventBridge event) writes the AMI ID to `/golden/app/latest`. CloudFormation uses `AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>` or `{{resolve:ssm:/golden/app/latest}}`
Launch template update	Distribution sets the new AMI as the launch template default version
AWS public parameters as parents	`/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64`, so recipes start from the latest Amazon Linux
Tags	Tag AMIs (`approved=true`, `version=1.4.0`) and look them up by tag

Replacing running instances

- **EC2 Auto Scaling instance refresh:** after the launch template has the new AMI, replace instances in batches with **minimum healthy percentage**, **checkpoints**, **skip matching**, and **automatic rollback** on CloudWatch alarms.
  - Automate it: **EventBridge** (image AVAILABLE), then **Lambda or Step Functions**, update the parameter or launch template, then `StartInstanceRefresh`.
- **CloudFormation:** update the stack with the new AMI parameter; the ASG `UpdatePolicy` (`AutoScalingRollingUpdate` or `AutoScalingReplacingUpdate`) replaces instances.
- **Elastic Beanstalk:** custom `ImageId` in `aws:autoscaling:launchconfiguration`; the configuration change triggers a rolling or immutable update.
- **CodePipeline:** an EventBridge rule starts the pipeline when an image is AVAILABLE: deploy to test, tests, approval, prod (instance refresh). Or a pipeline stage calls `StartImagePipelineExecution` when the application changes.

Infrastructure as code

- CloudFormation: `AWS::ImageBuilder::Component`, `ImageRecipe`, `ContainerRecipe`, `InfrastructureConfiguration`, `DistributionConfiguration`, `ImagePipeline`, `Workflow`, `LifecyclePolicy`.
- `AWS::ImageBuilder::Image` builds an image **during stack creation**; `Fn::GetAtt Image.ImageId` passes it to other resources.
- CDK and Terraform have the same resources.

Governance (rehydration pattern)

- **Rebuild on a schedule** (for example weekly, only when dependencies changed) and roll out with instance refresh.
- **AWS Config rules:** `approved-amis-by-id`, `approved-amis-by-tag`, `ec2-instance-managed-by-systems-manager`.
- **Inspector** also scans **running instances**: new CVEs mean rebuild the image.
- **Patch Manager** patches **long-lived** instances in place; **Image Builder plus replacement** is the immutable alternative.

Part 5 - Security points

- **IMDSv2** on build instances, and required in production launch templates.
- **Encrypt** AMIs and snapshots with **KMS**. Cross-account: grant target accounts (and the ASG service-linked role) use of a **customer managed key**.
- **No secrets in AMIs**: fetch them at runtime from Secrets Manager or Parameter Store.
- **Hardening: STIG** components or **CIS-hardened** parents, **Inspector scanning**, failing tests or a workflow `WaitForAction` gate on critical issues.
- **Least privilege:** minimal instance profile; separate roles for builds and distribution.
- **Private builds:** private subnets with **VPC endpoints**; internal package mirrors (for example **CodeArtifact**).

Part 6 - Image Builder vs alternatives

	EC2 Image Builder	Packer in CodeBuild	Manual / user data only
Management	Fully managed	You maintain templates and the build project	No image pipeline
Testing	Built-in **validate and test** phases, test instance	Your own scripts	None
Scanning	**Inspector** built in	Add yourself	No
Distribution	Multi-Region, multi-account, launch templates, SSM parameters	Scripted	No
Lifecycle cleanup	**Lifecycle policies**	Scripted	No
Exam view	**Least operational overhead**	Existing Packer investment, multi-cloud	Slow boots, drift

Part 7 - Exam traps

Trap	Truth
Build fails at the start, no component logs	**Instance profile** lacks `AmazonSSMManagedInstanceCore` / `EC2InstanceProfileForImageBuilder`, or **no route to SSM and S3** (no NAT or endpoints)
Open SSH (port 22) for Image Builder	Not needed. It uses **Systems Manager**: no inbound rules, no key pair
Need to inspect a failed build but the instance is gone	`terminateInstanceOnFailure: false`, plus CloudWatch Logs or S3 logs
The test phase runs on the build instance	**validate** = build instance. **test** = **new instance from the new AMI**
Rebuild only when the base image or components change	`EXPRESSION_MATCH_AND_DEPENDENCY_UPDATES_AVAILABLE`
Always use the newest Amazon Linux base	**AWS-managed parent with version filter** `x.x.x`, or the **public SSM parameter**
Edit an existing component version to fix a bug	Versions are **immutable**: create a **new version** and a new recipe version
Shared encrypted AMI can't be launched in another account	**KMS key policy** must allow the target account and its **Auto Scaling service-linked role**. Use a **customer managed key** (AWS-managed keys can't be shared)
Developers hard-code AMI IDs	Publish to **SSM Parameter Store**, or update the **launch template** from distribution
Roll out a new AMI to a running ASG safely	Launch template update, then **instance refresh** (min healthy %, checkpoints, alarm rollback)
Old AMIs and snapshots cost money	**Image lifecycle policies** (deprecate, disable, delete; snapshots too)
Human approval before distribution	Image Builder **workflow** `WaitForAction`, or **CodePipeline manual approval** after the AVAILABLE event
Deploy automatically when an image is ready	**EventBridge Image State Change (AVAILABLE)** to Lambda, Step Functions or CodePipeline
Bake the database password into the AMI	Never. Fetch secrets at runtime
Image Builder only makes AMIs	It also builds **container images** (container recipe to ECR)
Patch Manager vs Image Builder	Patch Manager = **in place** on running instances. Image Builder = **new images** for immutable replacement

Part 8 - Hands-on lab (about 60 to 90 minutes, low cost)

- **Cost note:** you pay for build and test instances while they run, AMI and snapshot storage, and Inspector scans. Delete everything at the end.

Lab 1 - The first golden AMI (console)

1. **IAM:** role `ImageBuilderInstanceRole` with `AmazonSSMManagedInstanceCore` and `EC2InstanceProfileForImageBuilder`, plus an instance profile.
2. **Component:** `install-nginx` version `1.0.0` from the YAML in Part 2.
3. **Recipe:** `golden-web` version `1.0.0`: parent Amazon Linux 2023 (AWS managed, `x.x.x`); components `update-linux`, `amazon-cloudwatch-agent-linux`, `install-nginx`; test component `simple-boot-test-linux`; encrypted gp3 root volume.
4. **Infrastructure configuration:** the instance profile, `t3.medium`, a subnet with NAT, **terminate instance on failure = false**, an SNS topic subscribed to your email.
5. **Distribution:** your Region, AMI name `golden-web-{{ imagebuilder:buildDate }}`, **SSM parameter** `/golden/web/latest`.
6. **Pipeline:** manual trigger, **image scanning on**. Run it and watch BUILDING, TESTING, DISTRIBUTING, AVAILABLE (about 30 to 40 minutes).
7. Check the new AMI, the **SSM parameter value**, the **Inspector findings** and the **CloudWatch Logs**.

Lab 2 - See a failure and debug it

1. Create `install-nginx` version `1.0.1` with a typo in the package name, update the recipe, and run again.
2. The build **FAILS**. Read the error in CloudWatch Logs. The instance is **still running**: connect with **Session Manager** and investigate.
3. Fix it in `1.0.2`. **Terminate the leftover instance manually.**

Lab 3 - Roll out with instance refresh

1. A launch template with AMI `resolve:ssm:/golden/web/latest` and an ASG with 2 instances.
2. Rebuild the image; the parameter now points to the new AMI.
3. Start an **instance refresh** (min healthy 50%, skip matching on) and watch instances being replaced.
4. **Automate:** EventBridge rule on `EC2 Image Builder Image State Change` with `state.status = AVAILABLE`, targeting a Lambda that calls `StartInstanceRefresh`.

Lab 4 - Schedule and lifecycle

1. Change the pipeline to a weekly cron with **only when dependency updates are available**.
2. Create an **image lifecycle policy**: keep the last 3 images, deprecate older ones, delete after 30 days including snapshots.

Lab 5 (optional) - Cross-account sharing

1. In the distribution, add launch permission for a second account (or your OU), with a **customer managed KMS key** whose policy allows that account.
2. In the second account, launch an instance from the shared AMI.

Clean up

- Delete the pipeline, recipes, components, infrastructure and distribution configurations, the ASG and launch template, the AMIs **and snapshots**, the SSM parameter, the SNS topic, the EventBridge rule and Lambda, and any leftover instances.

Part 9 - Check yourself

Questions

1. Name the main resources an image pipeline combines.
2. What is the difference between the validate and test phases?
3. A build fails before any component runs and there are no AWSTOE logs. Name the two most likely causes.
4. How do you keep a failed build instance for troubleshooting?
5. The pipeline runs weekly, but a new image should be built only when the base OS or a component changed. What setting?
6. How do CloudFormation stacks always launch from the latest golden AMI without editing templates?
7. A new AMI is ready. How do you replace instances in a running Auto Scaling group safely and automatically?
8. Account B can't launch an encrypted AMI shared from account A. What is the likely fix?
9. Old AMIs and snapshots keep piling up. What is the managed fix?
10. You need a manual security approval before an image is distributed. What are two options?
11. Patch Manager or Image Builder: when do you choose each?
12. Can Image Builder create Docker images? Where do they go?

Answers

1. **Image recipe** (parent image plus components), **infrastructure configuration**, **distribution configuration**, and the pipeline's **schedule / trigger** settings.
2. **validate** runs on the **build instance** before the AMI is created. **test** runs on a **new instance launched from the new AMI**.
3. The **instance profile** lacks `AmazonSSMManagedInstanceCore` / `EC2InstanceProfileForImageBuilder`, or there is **no network route to SSM and S3** (no NAT or endpoints).
4. `terminateInstanceOnFailure` **= false** in the infrastructure configuration, then connect with Session Manager.
5. Schedule start condition `EXPRESSION_MATCH_AND_DEPENDENCY_UPDATES_AVAILABLE`, with the parent version filter `x.x.x`.
6. Distribution writes the AMI ID to **SSM Parameter Store**; templates use `AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>` or `{{resolve:ssm:...}}`.
7. **EventBridge (image AVAILABLE)** to Lambda or Step Functions, update the launch template (or rely on the SSM parameter), then **ASG instance refresh** with min healthy %, checkpoints and alarm rollback.
8. A **customer managed KMS key** whose **key policy allows account B** and its Auto Scaling service-linked role.
9. **Image Builder lifecycle policies** (deprecate, disable, delete by count, age or tags, including snapshots).
10. An Image Builder **workflow with** `WaitForAction`, or a **CodePipeline manual approval** before rollout.
11. **Patch Manager:** patch long-lived instances **in place**. **Image Builder:** build new images and **replace** instances (immutable, preferred).
12. Yes. **Container recipes** build Docker images and distribute them to **ECR**, including other Regions.

Quick memory summary

- **Golden AMI** = baked, patched, hardened, versioned; immutable replacement; never bake secrets.
- **Components** (AWSTOE YAML: **build, validate, test**; immutable semantic versions) go into an **image recipe** (parent image with `x.x.x`, block devices). **Infrastructure configuration** (instance profile with **SSM core + EC2InstanceProfileForImageBuilder**, subnet with route to SSM/S3, `terminateInstanceOnFailure`, SNS, logs). **Distribution** (Regions, accounts/OUs, KMS, **launch template**, **SSM parameter**).
- **Pipeline:** manual, API or cron; **only on dependency updates**; tests; **Inspector** scanning; **workflows with WaitForAction**.
- **Lifecycle policies** clean up old AMIs and snapshots.
- **Rollout:** EventBridge AVAILABLE, then Lambda / Step Functions / CodePipeline, launch template or SSM parameter, then **ASG instance refresh**.
- **Cross-account encrypted AMIs** need a **customer managed KMS key** policy for the target account and its ASG service-linked role.